PROVEIT RESEARCH NOTE

Further Lean Proof Simplifications

Four directions after the existing proposal corpus

Prepared for Vladimir Reshetnikov

26 October 2026

Scope. The maintained Lean development in `Computability/DiophantineConsistency`, with all ideas in the six placed proof-simplification packets and five incoming archives treated as prior work.

Source pin. `6f46a5e3480a2fa2e45993b8090826f256aeaa56`.

Deliverables. Integration patches, complete changed Lean modules, source and exclusion manifests, exact compiler logs, replay scripts, and this report in PDF and LaTeX.

The useful remaining changes are about proof dependency direction: identify an existing codec, make local coefficient extraction foundational, remove sign cases from a Pell minimality proof, and isolate the least-exit argument shared by two geometric proofs.

Novelty in this report means absence from the audited proposal corpus. It does not mean a new mathematical theorem, a new Lean technique in general, or a claim that every possible refactoring has been exhausted.

Contents

1 Outcome, scope, and novelty standard **2**
1.1 The source snapshot . . . 2
1.2 Exclusion audit . . . 2
1.3 What is claimed . . . 3

2 Identify the list codec before proving more list lemmas **3**
2.1 The remaining duplication . . . 3
2.2 First consequence: reuse the standard length theorem . . . 4
2.3 Second consequence: derive typed parent checking from raw checking . . . 4
2.4 Why this survives the prior proposals . . . 4
2.5 Measured effect and integration order . . . 5

3 Make a local coefficient fibre the foundational theorem **5**
3.1 The mathematical statement . . . 5
3.2 Why the existing layering repeats work . . . 5
3.3 The checked interface . . . 6
3.4 Reuse the selective zero theorem for separated rows . . . 6
3.5 Relation to excluded work and measured effect . . . 6

4 Use the magnitude of the second Pell coordinate **7**
4.1 The precise target . . . 7
4.2 The shorter argument . . . 7
4.3 Optional general criterion . . . 8
4.4 Novelty boundary . . . 8

5 Separate first-exit minimality from tile geometry **9**
5.1 Two proofs with the same well-ordering step . . . 9
5.2 The common theorem . . . 9
5.3 Applications retain the exact geometric premises . . . 9
5.4 An explicit source-size tradeoff . . . 10

6 Measurements, formal checks, and reproducibility **10**
6.1 Count the complete cost of each abstraction . . . 10
6.2 What was compiled . . . 11
6.3 Axiom footprint . . . 11
6.4 How to replay and integrate . . . 12

7 Adoption order and bounded follow-up work **12**
7.1 Recommended first pass . . . 12
7.2 Two checked prototypes, with no unearned production saving . . . 13
7.3 What would change these recommendations . . . 13

References **13**

1 Outcome, scope, and novelty standard

This report proposes four additional directions. The first three produce local source reductions; the fourth makes two geometric proofs share one well-ordering argument, at a small increase in total source size. This distinction matters: a shorter application site is not automatically a shorter development once the supporting theorem is counted.

1. **Identify the existing list code with Mathlib's encoding.** Use that identity to obtain length and injectivity facts, and derive typed parent-list correctness from the raw checker theorem already present.
2. **Base quadratic coefficient extraction on a local pair fibre.** Prove the coefficient identity for an arbitrary weight map with exactly the two required contributing ordered pairs, then specialize to global Sidon and selective coding.
3. **Remove sign cases from the fundamental Pell-unit proof.** Work with the magnitude of the second coordinate, retaining the existing square-gap comparison. Keep the more general minimum-coordinate criterion as an optional checked abstraction.
4. **Factor the least-exit argument in the two strip-wall proofs.** Separate interval minimality from the asymmetric tile geometry on the left and right of Start.

1.1 The source snapshot

Repository access began by resolving the current main branch through GitHub. All source analysis and integration patches use the immutable commit `6f46a5e3480a2fa2e45993b8090826f256aeaa56`. Its recorded author time is 26 October 2026, 16:43:16 UTC. The commit already merges several earlier simplification adoptions, but the adoption table also records proposals that are pending or deferred [1, 2]. We do not interpret those statuses as permission to repeat them.

The task's premise is stronger than "find edits missing from main." We treat the mechanisms proposed by earlier reports as exhausted even where their code is absent, uncompiled, rejected, or only described as future work. The numerical before/after measurements are nevertheless against the real pinned files. They are not estimates for a hypothetical tree formed by simultaneously applying every mutually alternative historical design.

1.2 Exclusion audit

The audit inventories 329 files in the two requested exclusion directories and 400 members of the five incoming ZIPs. The six placed packets are `2026-10-26a` through `2026-10-26f`. All eleven principal manuscripts were inspected by section; supplementary proposal notes, patches, and the earlier reports' own novelty audits were also searched and reviewed. The archive contents were read as evidence, not executed.

The accompanying `evidence/exclusion-ledger.json` and `evidence/EXCLUSION_LEDGER.md` identify 60 prior technique families, with file-and-line or archive-member-and-line references. `evidence/prior-corpus-manifest.json` records their source hashes. Inventory coverage does not assert that every incidental copied source snapshot or binary received an independent semantic reading.

Examples of deliberately excluded ideas include scanner simulation, primitive-recursion congruence, decoder completeness, coefficient envelopes, sum-of-squares interfaces, generated circuit certificates,

support envelopes, finite invariant predecessor lemmas, carry uniqueness, rational-root descent, and aligned-cell additive equivalences. Some apparently fresh directions were explicitly rejected in the older manuscripts themselves; those too are excluded. The present four proposals were compared with their nearest prior mechanisms, not merely searched under their proposed names.

1.3 What is claimed

The supplied patches preserve the existing executable encoding and arithmetic definitions. New statements are proved, and existing public theorem types are retained. The formal checks concern proof simplifications; they do not establish an additional Diophantine equivalence, improve a variable count, or resolve a mathematical gap elsewhere in the research program.

The report separates three kinds of evidence:

- *Full modified module checks*: the actual source module is compiled with its replacement proof and production imports.
- *Focused checks*: exact theorem applications, helper checks, and axiom reports examine the changed interface.
- *Architectural recommendations*: a checked general theorem may support future reuse without any claim that a whole downstream migration has already been implemented.

All final counts and check outcomes are recorded in `evidence/results.json`. The validation section states the actual rebuild scope and the limits of the evidence.

2 Identify the list codec before proving more list lemmas

2.1 The remaining duplication

The wire-format module `AnnotatedCodeArithmetic` defines `listCode` on lists of natural numbers by

$$C([]) = 0, \qquad C(x :: xs) = \mathrm{Nat.pair}(x, C(xs)) + 1.$$

At the pinned Mathlib version, this is the same function as the installed `Encodable` instance for `List Nat`. The equations arise in different definitions, so the useful interface is a proved pointwise identity, not a change of the repository's definition [3, 4].

The distinction is valuable because the existing code is not merely a mathematical encoder. Supplied arithmetic programs, program-size statements, and downstream reductions refer to its literal implementation. Replacing the definition by a library alias would create an unnecessary definitional compatibility obligation. Adding one theorem leaves those data untouched.

The actual bridge is short:

```
theorem listCode_eq_encode (xs : List Nat) :
    listCode xs = Encodable.encode xs := by
  induction xs with
  | nil => rfl
  | cons x xs ih => exact congrArg (fun n => Nat.pair x n + 1) ih
```

The integration file keeps this as an explicitly invoked bridge. It does not globally redirect every simplifier traversal of executable code into the encoding typeclass.

2.2 First consequence: reuse the standard length theorem

`AnnotatedRename.length_le_tableCode`, in the module `AnnotatedRenameCode`, currently proves its length bound by a separate induction using a pairing inequality. A table is a list of row codes, so the bridge reduces the entire argument to the library's `Encodable.length_le_encode`:

```
simpa only [tableCode, listCode_eq_encode, List.length_map] using
  (Encodable.length_le_encode (rows.map rowCode))
```

This is stronger maintenance simplification than merely shortening the induction. There is now one source for the general claim that a list's length is bounded by this exact code. The row encoder requires no injectivity hypothesis for this application: the theorem concerns the length of the mapped list, and `List.length_map` supplies that equality [5, 4].

2.3 Second consequence: derive typed parent checking from raw checking

`AnnotatedParents` has both a typed-list theorem, `check_lists`, and a raw-code theorem, `check_raw_iff`. The former originally performs another induction over the expected list, with nested fuel and parent-list cases. The latter already reconstructs the complete parent spine from an arbitrary natural code. Its proof does not need `check_lists` [6].

Move `check_lists` below `check_raw_iff`, preserving its public statement, and specialize the stronger theorem to `listCode parents`. The only remaining issue is uniqueness of the decoded spine. The library encoding is injective, and the bridge provides precisely that fact for `listCode`:

```
rw [check_raw_iff fuel history expected (listCode parents) hf]
simp [listCode_eq_encode, Encodable.encode_injective.eq_iff]
```

Schematically, the raw theorem has the form

$$\text{check}(c) = 1 \quad \Longleftrightarrow \quad \exists ps,\ c = C(ps)\ \wedge\ \text{ParentsAgree}(ps).$$

Setting $c = C(parents)$ and applying injectivity removes the existential. The premise `expected.length ≤ fuel` is retained exactly. No stronger fuel assumption, parser termination argument, or exclusion of malformed codes has been inserted.

2.4 Why this survives the prior proposals

Earlier reports propose lookup-library reuse, generic decoder graphs, and using a decoder left inverse for code injectivity. Those are already assumed available. The additional fact here is an exact identification with an existing *library instance*, which simultaneously supplies a length bound and injectivity. The second change makes the typed result a corollary of the independently proved raw result. It does not introduce another scanner or decoder-correctness framework.

2.5 Measured effect and integration order

With all three modules included, the aggregate physical source length decreases from 461 to 453 lines, counting the new import, explanatory comment, and bridge theorem. The selected proof bodies go from 19 to 7 lines when the bridge proof is included. All three modified modules were compiled with their production imports.

There is one foundation-level tradeoff. The original table-length theorem depends on `propext` and `Quot.sound`; the library replacement also depends on `Classical.choice`, through the proof-bearing list encoding instance and its library lemmas. This is a standard Lean axiom, not a new assumption added to the project, but it is an observable change to `##print axioms`. The parent-checker theorem already has all three standard dependencies and does not increase its footprint. The table-length change is therefore supplied as a separate optional patch. Omitting it retains the original footprint and gives a four-line aggregate reduction for the codec bridge plus parent specialization alone.

Apply `01-list-encoding-core.patch`, which includes the codec bridge and parent theorem reordering. Apply `02-optional-table-length.patch` only if the extra axiom dependency is acceptable. Existing public signatures are unchanged. The remaining maintenance obligation is explicit: if the repository's encoding or Mathlib's chosen list encoding changes, the small bridge is where that incompatibility will surface. It should not be hidden by an overly broad simplification rule.

3 Make a local coefficient fibre the foundational theorem

3.1 The mathematical statement

Let I be a finite index type, R a commutative semiring, and $w : I \to \mathbb{N}$, $u : I \to R$. Define the sparse encoding

$$E_w(u) = \sum_{a \in I} u_a X^{w(a)}.$$

Fix $i, j \in I$. Suppose the selected exponent has exactly the expected ordered-pair fibre:

$$w(a) + w(b) = w(i) + w(j) \quad \Longleftrightarrow \quad (a = i \wedge b = j) \vee (a = j \wedge b = i). \tag{1}$$

Then

$$[X^{w(i)+w(j)}]E_w(u)^2 = \begin{cases} u_i^2, & i = j, \\ 2u_i u_j, & i \neq j. \end{cases} \tag{2}$$

This needs only the fibre of one exponent. The weight map may have arbitrary collisions elsewhere. It needs no positivity, cancellation, characteristic zero, integral-domain assumption, or ordering on R. In characteristic two the off-diagonal coefficient may vanish; equation (2) still states the correct identity.

3.2 Why the existing layering repeats work

The global development in `QuadraticCoding` proves coefficient extraction from the explicit cubic Sidon weights in `SidonPositions`. The selective development in `SelectiveQuadraticCoding` works with an arbitrary weight map and a local `Isolated` hypothesis. Both expand a square into a double sum and prove essentially the same diagonal/off-diagonal calculation [7, 8, 9].

The stronger global assumption is useful to establish equation (1) for *every* pair, but it is not needed by the coefficient calculation itself. The new low-level module `QuadraticFiber` states the calculation at its actual level of generality. This avoids making the global module import the selective module, which already depends on global notation and would create an import cycle. Both instead use a common foundation containing only the generic coefficient theorem.

3.3 The checked interface

The new theorem's hypothesis is local and exact:

```
theorem coeff_square_at_isolated
    [Fintype I] [DecidableEq I] [CommSemiring R]
    (w : I → Nat) (u : I → R) (i j : I)
    (hiso : ∀ a b, w a + w b = w i + w j ↔
      (a = i ∧ b = j) ∨ (a = j ∧ b = i)) :
    ((∑ a, Polynomial.monomial (w a) (u a)) ^ 2).coeff
      (w i + w j) = if i = j then u i ^ 2 else 2 * u i * u j
```

The proof expands the finite product once, rewrites the coefficient test by `hiso`, and separates the diagonal from the two distinct off-diagonal terms. Keeping the diagonal case explicit is essential: when $i = j$, the two descriptions of the ordered pair refer to one term, not two.

The global diagonal and off-diagonal theorems specialize this with `SidonPositions.weight_sum_eq_iff`. The global theorem on canonical unordered pairs uses the same specialization directly. The selective theorem becomes one application with its existing `Isolated` proof. The public definitions of `encoded`, `pairWeight`, and the row polynomials are preserved.

3.4 Reuse the selective zero theorem for separated rows

The same dependency correction has a useful second application. In `MultiQuadraticCoding`, the theorem `coeff_row_eq_zero_of_outside` repeats a nested polynomial expansion to show that a coefficient outside a row's permitted window is zero. The existing selective theorem `coeff_at_other_target` already proves this conclusion whenever each potential contributing exponent differs from the target [10, 8].

Specialize that theorem to the explicit Sidon weights and full pair support. The client then retains only the arithmetic reason for noncollision: the pair-weight bound, the bound on the sum of two weights, and the lower bound on the row target. Its closing `omega` call checks the interval arithmetic. There is no second sum/monomial proof in the client.

This is part of the same direction rather than a separate claimed novelty: general selective coefficient facts should sit below global special cases.

3.5 Relation to excluded work and measured effect

The earlier coefficient proposals concern upper envelopes, naturality, support bounds, finite push-forward sums, and degree certificates. Those are assumed to exist already. Here the new reusable statement is an *exact* coefficient formula from a two-point fibre, including the diagonal multiplicity rule, and the proof graph is reorganized around it.

The integration patch adds one 32-line helper module and changes five existing proof bodies across three modules. Aggregate source size is reported with that helper included in Section 6; blank separators between declarations are preserved. The entire four-module candidate was compiled, not merely the generic lemma in isolation.

The mathematical advantage outlasts a line-count reduction. A future weight construction needs to prove the local fibre condition at its chosen targets. It can then reuse the coefficient calculation even if unrelated pair sums collide. A new global Sidon theorem is unnecessary for this particular proof obligation.

4 Use the magnitude of the second Pell coordinate

4.1 The precise target

In `NativePellBootstrap`, the explicit solution

$$a = (2Q + 1, 2), \qquad d = Q(Q + 1), \qquad Q \in \mathbb{N},\ Q > 0$$

is proved fundamental for the Pell equation

$$x^2 - dy^2 = 1.$$

The statement `firstUnit_fundamental` is used by `first_norm_classification`; the latter turns actual norm solutions into powers of the fundamental unit [11].

The existing proof establishes $y^2 \geq 4$ for a competing solution with $x > 1$. Assuming $y^2 < 4$, it derives the alternatives $y = -1$, $y = 0$, and $y = 1$, then handles them separately. The two nonzero cases have the same norm equation. The zero case is already ruled out by a theorem in Mathlib's Pell API.

4.2 The shorter argument

For a competing solution $b = (x, y)$ with $x > 1$, use `Pell.Solution₁.y_ne_zero_of_one_lt_x`. If $y^2 < 4$, this gives

$$0 < |y| < 2.$$

Integer discreteness immediately gives $|y| = 1$, hence $y^2 = 1$. No sign of y has to be selected. Substitution into the norm equation yields

$$x^2 = Q(Q + 1) + 1 = Q^2 + Q + 1.$$

Because $Q > 0$,

$$Q^2 < x^2 < (Q + 1)^2.$$

Since $x > 1$, comparison of nonnegative squares implies $Q < x < Q + 1$, impossible for integers. Therefore $4 \leq y^2$. The norm identity now gives

$$(2Q + 1)^2 = 1 + 4Q(Q + 1) \leq 1 + Q(Q + 1)y^2 = x^2,$$

so $2Q + 1 \leq x$. Together with the positivity of the displayed solution, this is exactly the fundamental-unit condition.

The central replacement fragment is:

```
  have hy0 := abs_pos.mpr
    (Pell.Solution₁.y_ne_zero_of_one_lt_x hb)
  have hylt := abs_lt_of_sq_lt_sq
    (by omega : b.y ^ 2 < (2 : Int) ^ 2)
    (by norm_num : (0 : Int) ≤ 2)
  have hy : |b.y| = 1 := by omega
  have hsq : b.y ^ 2 = 1 := by nlinarith [sq_abs b.y]
```

The supplied production patch adds no helper or import. Only the proof of `firstUnit_fundamental` changes. Its public statement and the `firstUnit` definition are unchanged. The full file decreases from 671 to 663 physical lines, with normal blank separation retained.

4.3 Optional general criterion

The archive also supplies a checked architectural prototype:

```
theorem fundamental_of_minimal_y_sq
    (a : Pell.Solution₁ d)
    (hax : 1 < a.x) (hay : 0 < a.y)
    (hmin : ∀ b : Pell.Solution₁ d,
      1 < b.x → a.y ^ 2 ≤ b.y ^ 2) :
    Pell.IsFundamental a
```

Its proof transfers the inequality on second-coordinate squares through `a.prop_x` and `b.prop_x`, then compares positive first coordinates. The positivity of d follows from the existing positive nontrivial Pell solution. The hypothesis deliberately concerns y^2 for all competing solutions with $x > 1$; it does not silently assume that the competitor's y is positive.

For this single client, the direct patch has the better immediate cost. The generic criterion belongs in a shared module only after another client justifies its abstraction cost. The optional prototype's helper lines are not included in the claimed eight-line production reduction. Its separate square-gap helper is explanatory; a future integration should reuse or relocate the already existing private square-gap result in `MinimumNorm`, rather than maintain two equivalent long-term helpers [13].

4.4 Novelty boundary

Prior Pell proposals address parity, congruence up to sign, source-register facts, and rational-root descent. This proposal concerns the different minimality obligation defining the fundamental solution. It still applies after those other improvements. The mathematical fact about consecutive squares is elementary; the new contribution here is the source-specific removal of the signed enumeration and identification of the smaller proof obligation.

5 Separate first-exit minimality from tile geometry

5.1 Two proofs with the same well-ordering step

`WangTiles942InitialStrip.left_wall_exists` finds the first tile to the left of Start that is not the left initializer. Its right-hand analogue finds the first tile after the head that is not the right initializer. Both construct a `Nat.find` witness, prove a bound from a known failure, prove that every preceding index is still an initializer, and split between an immediate exit and an initializer predecessor [12].

The geometric data are different and should stay different. The left search starts at one and has a witness by N; the right search starts at two and has a witness by $N + 1$. The left seam is classified by east color two, while the right seam is classified by west color four. The proposed shared theorem captures only the well-ordering argument.

5.2 The common theorem

For a decidable predicate $P : \mathbb{N} \to \text{Prop}$, a starting index a, and a known failing index $b \geq a$, prove

$$\begin{aligned} \exists k, \quad & a \leq k \leq b, \quad \neg P(k), \\ & \forall j,\ a \leq j < k \implies P(j), \\ & k = a \quad \vee \quad P(k-1). \end{aligned}$$

The exact interface is:

```
theorem exists_first_failure (P : Nat → Prop) [DecidablePred P]
    (a b : Nat) (hab : a ≤ b) (hb : ¬ P b) :
    ∃ k : Nat, a ≤ k ∧ k ≤ b ∧ ¬ P k ∧
      (∀ j : Nat, a ≤ j → j < k → P j) ∧
      (k = a ∨ P (k - 1))
```

The disjunction is necessary. If the first position already fails, $k = a$, then $k - 1$ need not lie in the searched interval. When $a = 0$, natural subtraction truncates, making an unconditional predecessor conclusion particularly inappropriate. The focused checks include this zero-start immediate-exit boundary and a later-exit case.

The proof chooses the least index satisfying $a \leq j \wedge \neg P(j)$. Minimality yields the interior condition. If the chosen index differs from a, then $a \leq k - 1 < k$, so the same interior condition proves the predecessor case. Periodicity, finite cardinality, and tiles are absent from this theorem.

5.3 Applications retain the exact geometric premises

For the left wall, instantiate

$$P(j) := T(-j) = \text{initLeft}, \qquad a = 1, \quad b = N.$$

The original periodic-Start argument supplies $\neg P(N)$. For the right wall, instantiate

$$P(j) := T(j) = \text{initRight}, \qquad a = 2, \quad b = N + 1.$$

The original periodic-head argument supplies $\neg P(N+1)$.

The `k = a` branch uses Start or the head initializer respectively. The other branch uses the returned initializer predecessor. Actual horizontal matching then supplies the boundary edge color, and the existing local classification theorem identifies the seam. No uniqueness of Start, global rectangular tiling, or machine-run hypothesis is added.

5.4 An explicit source-size tradeoff

The client module decreases from 148 to 133 physical lines. The new production helper is 28 lines, so the aggregate changes from 148 to 161: *13 additional physical lines*. This is not a net shortening claim. The gain is that two copies of the least-exit proof become one theorem, and each geometric proof contains only its own boundary reasoning.

For a project optimizing immediate total source length, defer this patch. For a project expecting further initial-strip or reset-strip arguments, the shared invariant can be worthwhile. The archive keeps this patch separate, so that decision does not affect the other reductions.

This is distinct from the older finite-invariant predecessor proposal: that proposal derives backward preservation from an injective map of a finite invariant set. Here the shared mechanism is least-failure minimality on a bounded natural interval, with an explicit starting boundary. Neither finite injectivity nor an orbit-period argument enters the helper.

6 Measurements, formal checks, and reproducibility

6.1 Count the complete cost of each abstraction

The table counts whole affected production files against the pinned source. An added helper begins with zero lines on the baseline side. Physical lines include comments and blank lines. Code lines are nonblank lines after removing Lean line comments and nested block comments, while preserving string literals. The accompanying manifest records per-file SHA-256 hashes and byte counts.

Patch group	Before	After	Saved	Code saved
Codec bridge and parent specialization	360	356	4	6
Optional table-length specialization	101	97	4	4
Local coefficient fibre and three clients	413	393	20	25
Pell absolute-coordinate argument	671	663	8	8
Optional first-failure abstraction	148	161	-13	-3
All five patch groups	1,693	1,670	23	40

The coefficient group includes the entire 32-line new module. The first-failure group includes the entire 28-line new module. Optional architectural prototypes and validation files are not production savings. The complete source byte count changes from 76,582 to 75,864.

For immediate reduction with unchanged axiom dependencies of existing theorems, use the codec/parent, coefficient, and Pell patches. Their affected files change from 1,444 to 1,412 physical lines: **32 lines saved**, and 39 code lines saved. Adding the optional table-length patch changes the aggregate saving to 36 physical lines and 43 code lines. The modest totals are intentional: the report counts the reusable interfaces it introduces.

6.2 What was compiled

Validation used Lean 4.32.0, compiler commit `8c9756b28d64dab099da31a4c09229a9e6a2ef35`, and the repository's Mathlib pin `81a5d257c8e410db227a6665ed08f64fea08e997`. Required repository dependencies were built from the pinned source. The final combined overlay compiled all ten complete production modules:

Module	Result
`AnnotatedCodeArithmetic`	exit 0
`AnnotatedRenameCode`	exit 0
`AnnotatedParents`	exit 0
`QuadraticFiber` (new)	exit 0
`QuadraticCoding`	exit 0
`SelectiveQuadraticCoding`	exit 0
`MultiQuadraticCoding`	exit 0
`NativePellBootstrap`	exit 0
`FirstExit` (new)	exit 0
`WangTiles942InitialStrip`	exit 0

These checks include the original executable definitions and assertions contained in those modules. Separate files checked the zero-start and later-exit boundaries of the new first-failure interface, and printed the axiom dependencies of the changed and new declarations. Both old and new versions of the existing selected theorem declarations were available for the axiom comparison. The self-contained combined patch also passed `git apply --check` against the original source snapshot.

The exact source hashes, commands' exit statuses, elapsed times, and raw logs are retained in `evidence/final-validation.json` and `evidence/final-logs`. Elapsed times are execution receipts; they are not controlled performance measurements. This was a targeted module build, not a repository-wide rebuild, a proof-term-size study, or a claim that all reverse dependencies were re-elaborated.

6.3 Axiom footprint

Here S abbreviates the ordinary Lean dependency set

$$S = \{\texttt{propext},\ \texttt{Classical.choice},\ \texttt{Quot.sound}\}.$$

Declaration group	Baseline	Proposed
Parent `check_lists` and unchanged `check_raw_iff`	S	S
Five existing coefficient declarations	S	S
Pell `firstUnit_fundamental`	S	S
Left and right wall existence	S	S
Table `length_le_tableCode`	$S \setminus \{\texttt{Classical.choice}\}$	S
New codec, coefficient, and first-failure helpers	—	S

No new axiom is declared. None of these inspected final declarations depends on `sorryAx`. The optional table-length change is the one observed increase in an existing theorem's axiom dependency set. Keeping its original proof avoids that increase. The new list bridge itself uses the library's proof-bearing encoding instance, even though the project encoding remains the same executable definition.

6.4 How to replay and integrate

The archive contains targeted patches, reference after-files under `overlay/`, and a portable `validation/replay.py` script. The baseline hashes identify the exact files used for the measurements. Replay requires a checkout with the recorded Lean toolchain and Mathlib pin; the script checks those baseline source hashes and builds the required modules, then compiles the overlay into a separate output directory. The baseline source files are not overwritten by replay. The prepared-environment replay mode was exercised successfully, including both optional prototypes. The ordinary Lake launcher is also supplied for prepared checkouts; it was not the mode used for the recorded run.

Lean resolves a top-level module namespace through one search-path directory. Consequently, the replay script stages unchanged dependency artifacts in a complete namespace alongside the new artifacts; merely prepending a sparse overlay directory would be insufficient. This implementation detail matters for reliable replay but does not alter any theorem statement.

For a branch where all previous proposals have already been applied, rebase the relevant patch hunks onto that branch. The full after-files are evidence of the pinned experiment, not instructions to replace files wholesale and restore unrelated older proofs. A patch that no longer applies cleanly may need a routine proof-level adaptation to the already-refactored interface. The corpus-relative ideas survive that situation; the exact line totals still describe only the pinned experiment.

7 Adoption order and bounded follow-up work

7.1 Recommended first pass

The coefficient patch is the strongest immediate candidate. It removes repeated mathematical expansion, applies under weaker local hypotheses, serves five existing proof sites, and saves 20 physical lines after the shared theorem's full cost. Its new module is small and has an explicit interface. Review the diagonal multiplicity and the selected exponent's fibre condition first; they are the essential correctness points.

The Pell patch is the smallest isolated proof edit. It adds neither an import nor a helper and saves eight lines. Its mathematical obligation is unchanged: the same interval between consecutive squares rules out the small second coordinate. The change is to derive that coordinate's square once from its absolute value. It is a suitable independent integration commit.

The codec/parent patch is useful where preserving the literal wire format is a priority. It makes an exact relationship to a library API explicit and turns an already available raw correctness theorem into the source of the typed correctness theorem. The four-line net saving is smaller, but the duplicate induction disappears. The optional length proof can be considered separately after examining its added `Classical.choice` dependency.

Defer the first-failure patch if immediate aggregate source length is the only criterion. Adopt it when shared control of interval minimality is valuable, or when an additional strip argument would otherwise introduce another copy. No arbitrary threshold on a predicted future saving is needed: compare the actual next client against the 28-line helper already supplied.

7.2 Two checked prototypes, with no unearned production saving

Mixed products. The optional `PairFiber.lean` file supplies a related local coefficient theorem for a product of two encodings. At the chosen exponent the two surviving contributions are $u_i v_j$ and $u_j v_i$; on the diagonal there is only $u_i v_i$. It can support future bilinear clients that should not be forced through a square. This prototype was compiled, but no present production client is migrated to it and no savings are counted for it. Integrate only when a concrete client justifies the extra interface.

Pell minimum-coordinate criterion. The optional `PellMinimum.lean` file proves that minimizing y^2 among solutions with $x > 1$, together with the candidate's positivity, suffices for fundamentality. It makes the mathematical proof obligation reusable while allowing competitors of either sign in y. For the one current target, the direct eight-line reduction is preferable. A second fundamental-unit proof would give a concrete reason to revisit the shared criterion.

Both prototypes are accompanied by successful compiler logs. They are separate from the ten-module production overlay and from all source-saving totals. Neither is a reason to reopen an already-excluded historical idea.

7.3 What would change these recommendations

Three concrete developments could affect the choice. A changed Mathlib list-encoding instance would require rechecking the exact codec bridge. Another polynomial encoding with only selected isolated targets would increase the value of the local coefficient theorem. Another strip-wall proof would provide a real third client for the first-failure abstraction. Each can be assessed through the supplied interfaces without redesigning the executable reductions.

The archive is meant to make those decisions reviewable: each direction has an exact target, a mathematical reason for the simplification, a separate integration patch, complete affected source, and a declared verification scope. The exclusion ledger accompanies the proposals so that later work can continue from this boundary instead of rediscovering the same mechanisms.

References

[1] Vladimir Reshetnikov, *ProveIt*, pinned source commit `6f46a5e3480a2fa2e45993b8090826f256aeaa56`, 26 October 2026. Repository source references below use this immutable pin.

[2] ProveIt, *Lean proof simplification: adoption record and six placed packets*. Placed proposal corpus; incoming proposal corpus. The accompanying manifest inventories the reviewed artifacts and hashes.

[3] ProveIt, `AnnotatedCodeArithmetic.lean`. The list-code definition and its arithmetic interface.

[4] Mathlib, `Mathlib/Logic/Equiv/List.lean`, commit `81a5d257c8e410db227a6665ed08f64fea08e997`. The list encoding and `Encodable.length_le_encode`; the relevant encoding equations were checked with this pin. General encoding injectivity is defined in `Mathlib/Logic/Encodable/Basic.lean` at the same pin.

[5] ProveIt, `AnnotatedRenameCode.lean`. Table encoding and `AnnotatedRename.length_le_tableCode`.

[6] ProveIt, `AnnotatedParents.lean`. The typed and raw parent-checker correctness theorems.

[7] ProveIt, `QuadraticCoding.lean`. Global quadratic coefficient identities.

[8] ProveIt, `SelectiveQuadraticCoding.lean`. Isolated targets and `coeff_at_other_target`.

[9] ProveIt, `SidonPositions.lean`. Explicit Sidon weights and the pair-sum classification.

[10] ProveIt, `MultiQuadraticCoding.lean`. Separated rows and out-of-window coefficient vanishing.

[11] ProveIt, `NativePellBootstrap.lean`. `firstUnit`, `firstUnit_fundamental`, and norm classification. The library API is in Mathlib's `Mathlib/NumberTheory/Pell.lean` at the same Mathlib pin.

[12] ProveIt, `WangTiles942InitialStrip.lean`. Left and right initializer-strip wall existence.

[13] ProveIt, `MinimumNorm.lean`. The private `not_isSquare_between` lemma, a possible shared source for the square-gap obligation in future generalizations.